lowThrust — how it works

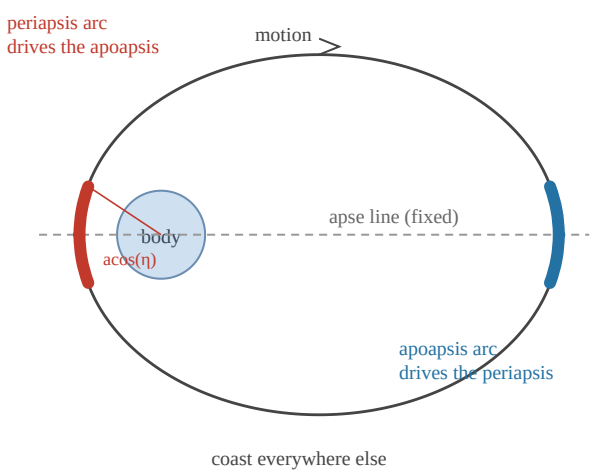
Low-thrust transfers -- within one SOI, to a moon, to another planet: the solver, its phase machine and the stand-in solar system. No kRPC side yet.

What it does

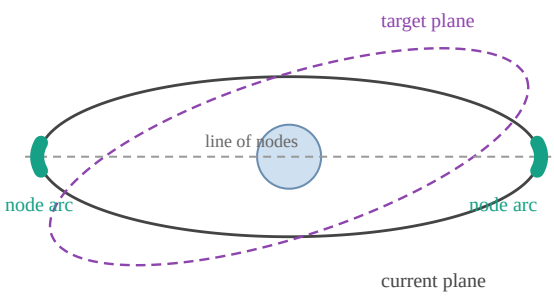
Given the vessel's current orbit, a target apoapsis, periapsis, inclination and longitude of the ascending node (anything left out is free) and optionally a time budget, it decides every tick whether to thrust, which way (a combination of prograde/retrograde and normal/anti-normal) and how hard, until each targeted apsis is within `TOLERANCE_M` (100 m) and the plane within `PLANE_TOLERANCE_DEG` (0.01°). The decision uses only the current orbit; there is no stored schedule, so a lost tick or a weak pass is corrected on a later pass by the same law.

module	role
`orbit.py`	elements of any conic, exact propagation (universal variables), time to periapsis, Lambert, radius toward a fixed direction, the impulsive references
`guidance.py`	the law: `lock` (apse line and target plane), `command`, `arrived`
`plan.py`	flies the law in the simulator to pick the arc width that meets the deadline
`sim.py`	the single-body stand-in: point mass, constant-thrust engine, spherical planet; the shared RK4 step and tick
`bodies.py`	the stand-in solar system: stock bodies on Keplerian rails, spheres of influence
`transfer.py`	targeting: closest approach and B-plane miss, mid-course correction, moon push, departure window, parking orientation, capture design, Lambert re-target, SOI hazards
`mission.py`	the phase machine of a whole transfer in patched conics; routing and leg chaining
`tools/simfly.py`, `tools/simmission.py`	command lines: one SOI with a deadline; a whole transfer with its timeline and burn list

The law: the Hohmann transfer in many small burns



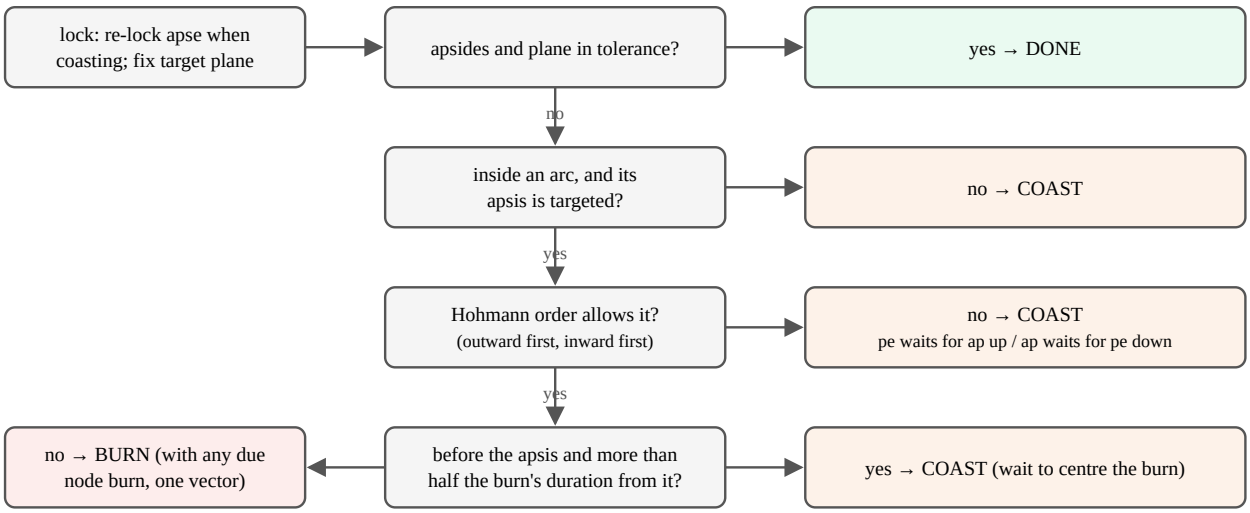
- **Decoupled.** A tangential burn at periapsis moves the apoapsis and, to first order, not the periapsis; one at apoapsis moves only the periapsis. So the periapsis arc drives the apoapsis to its target and the apoapsis arc drives the periapsis. The last pass nulls its element exactly.
- **Radius toward a fixed direction.** The quantity driven is the orbit's radius toward the far end of the apse line, `r = p / (1 + e·u)`, smooth through circular. Its rate per m/s of prograde is differenced numerically (`RATE_DV`).



Two arcs, centred on a fixed apse line. Each fixes the opposite apsis.

- **Arc half-width** is `acos(η)`; η is a tangential burn's effectivity off-apsis. Default η = 0.99 (±8°).
- **Prograde or retrograde** for an apsis burn, so plain SAS modes can hold it; a burn combined with a plane change points between prograde and normal.

What triggers what, each tick



One tick of the apsis half of `guidance.command`; the node half (next section) runs alongside. "need" is the burn time at full thrust to null the driven radius.

- **Apse line.** Held through each burn; re-locked to the osculating periapsis whenever the vessel is coasting. On a circular orbit with no line yet, the vessel's position sets it.
- **Burn timing.** A burn begins half its remaining duration before the apsis and runs at full throttle until its radius is on target or the arc ends; the final tick's throttle is the fraction still needed. No pass is longer than its arc.
- **Sign.** Prograde if raising the far radius closes the error, retrograde otherwise. The acceleration used is the one the engine gives now, never the rated figure.

Plane change

- **Target plane** fixed once (`lock`). With `lan` given, from inclination and node longitude. With `lan` free, the current node line is kept; an equatorial start has none, so the plane is fixed when the plane change may begin, with its nodes on the apse line of that moment.
- **Node arcs** of the same half-width `acos(η)`, burns starting half their duration before the node. Normal thrust tilts the orbit normal along −T·sign, so the sign turns it toward the target plane. Burn time needed: δ·h / (r_{node}·accel).

Normal/anti-normal arcs at the two nodes with the target plane.

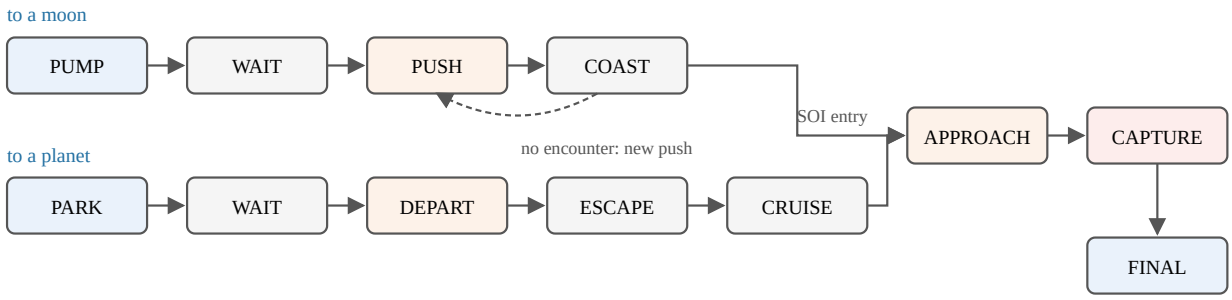
- **The high node.** A node whose radius is below η of the other's is skipped: a plane change costs the horizontal velocity it turns, least where the vessel is slowest.

gate	effect
apoapsis still being raised	no plane burns yet: the plane is changed on the biggest orbit
plane still to change	the apoapsis is not lowered
periapsis still to change	plane burns only at the node on the apoapsis side, where they overlap the periapsis burns
apsis arc and node arc both due	one burn: direction (prograde, normal) in proportion to the seconds each still needs, started half the combined duration early, throttle min(1, combined need / tick)

The planner and the time budget

Narrow arcs are many small, near-impulsive burns: cheapest and slowest. Wide arcs deliver more per pass and finish sooner for more dv. `plan.plan` flies the law in the simulator at `PLAN_GRID` (13) arc widths spaced evenly in half-width between `ETA_MAX` (0.999, ±2.6°) and `ETA_MIN` (0.2, ±78°), and takes the narrowest whose flight arrives within the budget. Without a budget it flies `ETA_DEFAULT` (0.99). If no width meets the budget it reports so and returns the fastest transfer. The prediction (dv, time, xenon, burn list) is the law itself flown forward, so it describes what the vessel will fly.

Transfers between bodies: the phase machine



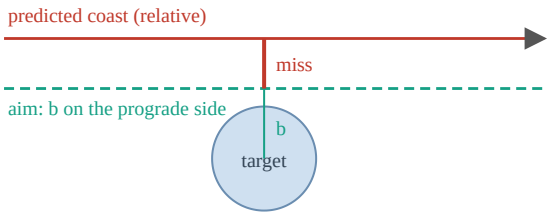
Every orbit-shaping phase (PUMP, PARK, FINAL) is the within-SOI law; TRIM (a phasing burn at periapsis) and an evasive burn in an unplanned SOI can interrupt any phase. A planet's moon is two legs: the planet leg's FINAL is the moon leg's waiting orbit. Routes: within one SOI; to a child (moon); to a sibling (planet); to a sibling's moon. Arrival requires the destination's SOI.

The phases of a transfer (`mission.Mission`), each re-decided every tick from the current state.

phase	what it does	ends when
PUMP	within-SOI law to the waiting orbit: in the moon's plane, apoapsis 0.6 of the way (or short of the moon's SOI), periapsis as low as possible but high enough for this engine to capture in one pass and outside other moons' bands	the waiting orbit is reached
WAIT (moon)	once a revolution, ahead of periapsis: solve the prograde push there that puts a direct encounter (within 0.75 revolution) on the aim point; skip the pass if its coast meets another moon	a push exists and periapsis is half its duration away

PUSH	prograde on a ledger (planned dv less delivered), refined by local re-solves	the ledger is spent
COAST / CRUISE	B-plane corrections when one needs more than 0.5 m/s, rechecked every tenth of the time left; to a moon, no encounter goes back to WAIT; to a planet, a miss beyond min(20 SOI, 1% of the target's distance) or a correction over 150 m/s is re-planned as a Lambert arc to the aim point	SOI entry
PARK	within-SOI law with a held apse line to the parking ellipse oriented on the window's departure asymptote	reached
WAIT (planet)	coast (in one step when the orbit cannot reach a boundary) to the periapsis pass nearest the window; skip a pass whose escape meets a moon	half the push duration before that periapsis
DEPART	prograde until the hyperbolic excess is the window's, or 60° past periapsis	escape (still bound: back to WAIT for another pass)
APPROACH	nudge the hyperbola's periapsis onto the capture altitude along its gradient	half the capture burn before periapsis
CAPTURE	retrograde, plus a periapsis-gradient term if the periapsis falls below the safe floor	bound below the capture apoapsis (fails if unbound 60° past periapsis)
FINAL	within-SOI law to the target orbit about the destination	arrived

Targeting



b = rp sqrt(1 + 2 mu / (rp v_inf^2))
miss = across-track offset - b (u x n)
dv = -J^T (J J^T)^-1 miss (J numerical, 2 x 3)

The B-plane miss and the minimum-norm correction.

- **Closest approach** of the exact two-body coast to the target on its rails: a coarse scan, then golden section. A minimum at the window's edge is no encounter.
- **Aim point**: the impact parameter *b* for the capture periapsis, on the side whose passage is prograde about the target's orbit normal.
- **Correction**: numerical Jacobian of the two miss components against the three impulse axes, minimum-norm solve. With a slow-arrival limit, the arrival speed is a third row and the solve is square.
- **Moon push**: the smallest prograde impulse at periapsis whose signed closest-approach distance equals *b* (scan, then bisection).
- **Re-target**: Lambert from the current state to the aim point over arrival times 0.6–1.6 of the planned time left, cheapest in impulse plus arrival speed; the arrival time is held while burning.

Departure, capture and hazards

- **Window**: Lambert over a grid of departure dates (one synodic period from the pump's estimated end) and flight times (0.5–1.5 of the Hohmann's), least $|v_\infty$ out$|$ + $|v_\infty$ in$|$, refined three times.
- **Parking ellipse**: periapsis above the atmosphere, apoapsis 0.9 of the highest clear of the planet's moons. Its plane contains the departure asymptote; of that family, the one whose plane change from the current orbit (at its higher node, on the ellipse) is cheapest. Periapsis direction $P = \cos \nu_\infty \, \hat{u} - \sin \nu_\infty \, (n \times \hat{u})$, with $\cos \nu_\infty = -1/e$ of the escape hyperbola.
- **Capture design**: the lowest capture periapsis (≥ the target's, ≥ 30 km above the atmosphere) at which this engine sheds the arrival speed into the loosest allowed ellipse within 80% of the time spent within ±60° of periapsis; the preferred apoapsis is 0.3 SOI under the moons, a looser one is tried if needed. A moon transfer raises the waiting periapsis until this is possible (a slower arrival). A planet transfer may cut the arrival speed in the cruise by up to 30%; beyond that the mission is refused at planning with the acceleration that would capture.
- **Hazards**: at the first tick of each arc burn the pass dv is capped at the largest fraction whose next revolution stays out of every other moon's SOI; once a revolution a phasing trim of ±0.5–8 m/s at periapsis if the coming one would meet a moon; an evasive burn along the periapsis gradient inside an unplanned SOI with the periapsis below the safe floor; after a slingshot, braking until the apoapsis is inside 0.6 SOI, then the orbit-shaping phase again.

The stand-in simulator

- Two-body gravity about Kerbin (μ 3.5316e12, radius 600 km, atmosphere 70 km); a vessel of dry mass, propellant, thrust and Isp (default 4 t + 1 t xenon, one 2 kN / Isp 4200 engine).
- A tick is period / `TICKS_PER_ORBIT` (180). Guidance is called once a tick and its command held. Coasting ticks are propagated exactly; thrusting ticks by `THRUST_SUBSTEPS` (4) RK4 steps, mass falling at thrust / (Isp g0).
- A flight ends on arrival, escape, periapsis inside the atmosphere, empty tanks, or `MAX_ORBITS`.
- **Ticks are uniform in angle**: r²/h per 2°, capped at period/180 -- near periapsis of a long ellipse a tick is seconds.
- **Patched conics** (`mission.py`): the vessel's state is relative to the body whose SOI it is in. A coasting tick is propagated exactly and bisected to the boundary when it crosses one (out to the parent, or into any child); the frame then switches by the body's state on its rails. Coasts that cannot reach a boundary are taken in one step.